

RTOS Final Project Overview

Application 2: Multi-Task Defense Scheduling System Overview

This project implements a real-time, multi-task defense system on an ESP32-S3 using FreeRTOS. The main purpose of the project is to demonstrate how multiple periodic tasks with different timing requirements can be scheduled, monitored, and analyzed on a dual-core embedded processor. The application uses four defense-themed tasks that all run on Core 1, while Core 0 is reserved for system monitoring through either a serial terminal or a web-based interface.

The four tasks represent different levels of urgency within a defense system. The highest-priority task, Threat Sensor Polling, runs every 10 milliseconds and simulates a fast sensor-processing operation using a fixed-point IIR filter. The second task, Fire-Control Update, runs every 20 milliseconds and performs statistical calculations on simulated sensor data. The third task, Telemetry CRC Framing, runs every 50 milliseconds and calculates a CRC-32 checksum over a telemetry packet to represent data integrity checking. The lowest-priority task, Mission Log Housekeeping, runs every 100 milliseconds and performs an insertion sort to simulate background logging and maintenance work.

The task priorities follow the rate-monotonic scheduling principle. Under this method, the task with the shortest period receives the highest priority because it must run most frequently. Therefore, the 10-millisecond sensor task has the highest priority, while the 100-millisecond housekeeping task has the lowest priority. When a higher-priority task becomes ready, it can preempt a lower-priority task that is currently running. This ensures that the most time-critical operations are serviced first.

Each task uses `vTaskDelayUntil()` to maintain a consistent periodic schedule. Unlike a normal delay, this function prevents timing drift by scheduling each execution relative to the task's original release time. After completing one iteration, each task increments a heartbeat counter. These heartbeat values allow the monitor to confirm that all tasks are still running. A counter that stops increasing may indicate that a task is blocked, starved, or no longer executing properly.

The program also measures the execution time of every task using the `MEASURE_WCET` macro. This macro records the start time, executes the task workload, measures the elapsed time, and stores the largest observed value. This value represents the task's measured worst-case execution time, or WCET. The WCET values can be compared to the task periods to calculate processor utilization and determine whether the complete task set is schedulable.

The ESP32-S3's two cores are used to isolate the timing-sensitive workloads from the monitoring system. The four defense tasks are pinned to Core 1, while the serial monitor or HTTP server runs on Core 0. This reduces interference with the WCET measurements and prevents Wi-Fi or terminal output from significantly affecting the real-time tasks.

The project supports two output modes. In terminal mode, a monitoring task prints each task's name, period, priority, heartbeat count, and maximum measured WCET once per second. In web-server mode, the ESP32 connects to Wi-Fi and displays the same information on an automatically refreshing web page.

Overall, this project demonstrates the major concepts involved in designing a real-time embedded system, including periodic tasks, fixed-priority scheduling, preemption, dual-core task isolation, heartbeat monitoring, WCET measurement, and schedulability analysis.